САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Домашнее задание №2 «Проектирование и технический дизайн API»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать API для фото-соцсети. Описать все функциональные и нефункциональные требования. Выбрать архитектурный стиль (REST API). Создать OpenAPI-спецификацию для всех эндпоинтов с указанием методов, параметров, тел запросов/ответов, кодов ошибок.

Ход работы

1. Функциональные требования

- **Аутентификация и регистрация**
  – Регистрация пользователя (username, email, password)
  – Логин с выдачей JWT-токена
  – Загрузка аватара пользователя
  – Получение профиля текущего пользователя
- **Управление постами**
  – Создание поста (загрузка фото высокого разрешения, автоматическое создание превью)
  – Получение ленты новостей (посты пользователей, на которых подписан)
  – Получение одного поста (с комментариями)
  – Удаление своего поста (с физическим удалением файлов)
- **Взаимодействие с постами**
  – Поставить/убрать лайк
  – Добавить/удалить пост в избранное
  – Добавить комментарий (текст + опциональное фото, сжатие до 4 Мп)
  – Удалить свой комментарий (с удалением прикреплённого файла)
  – Получить все комментарии поста
- **Социальные связи**
  – Подписаться на пользователя / отписаться
  – Получить уведомления (лайки, комментарии, подписки)
  – Отметить уведомление как прочитанное

2. Нефункциональные требования

- **Безопасность**
  – Пароли хранятся в хешированном виде (bcrypt)
  – JWT-токен с истечением срока действия (7 дней)
  – Все действия, кроме регистрации и логина, требуют валидного токена в заголовке Authorization: Bearer <token>
  – Пользователь может удалять только свои посты/комментарии
- **Производительность**
  – Обработка изображений асинхронно (sharp)

– Для постов создаётся превью (ширина 1200px), чтобы уменьшить трафик в ленте
– Оригинал фото поста доступен только при запросе конкретного поста

- **Хранение**
  – Файлы хранятся на диске в папке uploads/ (раздельно для аватаров, постов, фото комментариев)
  – База данных – SQLite (легковесная, не требует отдельного сервера для ЛР1)
  – При удалении ресурса соответствующие файлы удаляются из файловой системы
- **Ошибки**
  – Возврат стандартных HTTP-кодов: 200, 201, 400, 401, 403, 404, 409, 500
  – Тело ответа содержит поле message с пояснением

3. Выбор архитектурного стиля

- **REST API** (без BFF, так как монолитное приложение).
- Ресурсы: users, posts, comments, likes, favorites, follows, notifications.
- Используются HTTP-методы: GET (чтение), POST (создание), PUT (обновление), DELETE (удаление).
- Формат данных – JSON (кроме загрузки файлов, где используется multipart/form-data).

4. Проектирование эндпоинтов (таблица)

Метод	Эндпоинт	Тело запроса	Тело ответа (успех)	Коды ошибок
POST	/api/auth/register	{username, email, password}	{id, username, email}	400 (не все поля), 409 (дубликат)
POST	/api/auth/login	{email, password}	{token, user: {id, username, email}}	400 (поля), 401 (неверные данные)
GET	/api/users/profile	–	{id, username, email, avatarPath, createdAt}	401, 404
POST	/api/users/avatar	multipart/form-data: avatar	{avatarPath}	400 (нет файла), 401
POST	/api/users/follow/:followingId	–	{message: "Followed"}	400 (сам на себя), 401, 404, 409 (уже

				подписан)
DELETE	/api/users/follow/:followingId	–	{message: "Unfollowed"}	401, 404
POST	/api/posts	multipart/form-data: photo, description?	{id, userId, originalPhotoPath, previewPhotoPath, description, createdAt}	400 (нет photo), 401
GET	/api/posts/feed	–	[Post] (массив постов подписок)	401
GET	/api/posts/:id	–	Post + вложенные user и comments	401, 404
DELETE	/api/posts/:id	–	{message: "Post deleted"}	401, 403 (не свой), 404
POST	/api/posts/:id/like	–	{message: "Liked"}	401, 404, 409 (уже лайк)
DELETE	/api/posts/:id/like	–	{message: "Unliked"}	401, 404
POST	/api/posts/:id/favorite	–	{message: "Added to favorites"}	401, 404, 409 (уже в избранном)
DELETE	/api/posts/:id/favorite	–	{message: "Removed from favorites"}	401, 404
POST	/api/comments/posts/:postId/comments	multipart/form-data: text, photo?	{id, userId, postId, text, photoPath, createdAt}	400 (нет text), 401, 404
DELETE	/api/comments/comments/:id	–	{message: "Comment deleted"}	401, 403 (не свой), 404
GET	/api/comments/posts/:postId/comments	–	[Comment] (с вложенным user)	401, 404
GET	/api/notifications	–	[Notification] (сортировка по убыванию даты)	401
PUT	/api/notifications/:id/read	–	{message: "Marked as read"}	401, 404

5. OpenAPI спецификация (YAML)

Код спецификации OpenAPI будет приложен в файле openapi.yaml, так как вставка 500 строк в отчёт будет выглядеть слишком некрасиво.

6. Примеры запросов и ответов (выборочно)

Регистрация:
POST /api/auth/register
Content-Type: application/json

{"username":"bogdan","email":"bogdan@example.com","password":"123456"}

Ответ:
{"id":1,"username":"bogdan","email":"bogdan@example.com"}

Создание поста (curl):
curl -X POST /api/posts -H "Authorization: Bearer <token>" -F "photo=@image.jpg" -F "description=Фотка)"

Ответ:
{"id":5,"originalPhotoPath":"/uploads/posts/original_...jpg","previewPhotoPath":"/uploads/posts/preview_...jpg","description":"Фотка)"}

Ошибка при повторном лайке:
POST /api/posts/1/like
Authorization: Bearer <token>

Ответ:
{"message":"Already liked"}

Вывод

В ходе выполнения ДЗ2 спроектировано REST API для фото-соцсети, охватывающее все функциональные требования: регистрацию, аутентификацию, управление постами, комментариями (с возможностью прикрепления фото), лайками, подписками, избранным и уведомлениями. Определены нефункциональные требования (безопасность, производительность, обработка ошибок). Составлена OpenAPI-спецификация (YAML), которая полностью описывает эндпоинты, форматы запросов и ответов, коды ошибок. Данная спецификация готова для составления документации и последующей реализации в рамках лабораторной работы №1.